

SMART RUMMY
AN EXPLAINABLE AI AGENT FOR INDIAN RUMMY
USING SEARCH AND HEURISTIC DECISION-MAKING

A PROJECT REPORT

for

Foundations of Artificial Intelligence

Submitted by

240701217 – JOBESON JULIUS

240701388 – PRAGADESH S



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report “**SMART RUMMY: AN AI AGENT FOR INDIAN RUMMY USING SEARCH AND HEURISTIC DECISION-MAKING**” is the bonafide work of “**JOBESON JULIUS (240701217) and PRAGADESH S (240701388)**” who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
1	INTRODUCTION	1
2	LITERATURE REVIEW	3
3	PROPOSED SYSTEM	6
	3.1 ARCHITECTURE DIAGRAM	8
4	MODULES DESCRIPTION	11
	4.1 MODULE 1: GAME ENVIRONMENT AND RULES ENGINE	11
	4.2 MODULE 2: INTELLIGENT AGENT AND COACH	12
5	IMPLEMENTATION AND RESULTS	17
	5.1 EXPERIMENTAL SETUP	17
	5.2 RESULTS	17
6	CONCLUSION AND FUTURE WORK	24
7	REFERENCES	25

ABSTRACT

Indian Rummy is a popular game of skill, but its digital versions focus on real-money play. They let a player move without ever explaining *why* a move is good, and their computer opponents are opaque. This project, **Smart Rummy**, is a web application in which a human plays 13-card Indian Rummy against one to three computer agents while an AI coach explains every decision in plain language.

Each computer player is designed as a utility-based intelligent agent that works under partial observability, using classical AI techniques only. Declarations are verified by backtracking search with failure memoisation. A hand is evaluated by a memoised bitmask dynamic program that finds the arrangement with the least deadwood. Actions are chosen by simulating every possible discard and ranking the outcomes with progress-cost and connectivity heuristics, and the open card is judged by one-step lookahead. Exact probabilities over unseen cards and inference from opponents' pickups complete the agent. Three agent designs give Easy, Medium and Hard levels.

The system uses React and FastAPI and passes 123 automated tests. A hand is evaluated in about 23 ms and a full agent turn takes about 150 ms. In simulation, Medium and Hard agents finish rounds in 26 turns on average against 38.5 for Easy, confirming a measurable difference in strength.

CHAPTER 1

INTRODUCTION

Card games have long served as test beds for artificial intelligence because they combine clear rules with uncertainty, hidden information and interaction with other players. Indian Rummy is one of the most widely played card games in India and is recognised as a game of skill. Each player holds 13 cards from two standard decks and two printed jokers, and must arrange all of them into valid combinations: *sequences* (three or more consecutive cards of one suit) and *sets* (three or four cards of the same rank in different suits). A valid declaration needs at least two sequences, and at least one of them must be *pure*, that is, formed without any joker. At the start of each round a card is cut from the deck, and every card of that rank becomes a *wild joker* that can replace any missing card.

On every turn a player draws one card, either from the face-down stock or from the face-up discard pile, and then discards one card or declares. Each turn therefore involves two decisions under uncertainty: whether the open card is worth taking, and which of fourteen cards to give up. Good play depends on evaluating the hand correctly, remembering which cards are already gone, estimating which unseen cards would help, and inferring what the opponents are collecting. In AI terms these are state evaluation, belief tracking, probabilistic reasoning and opponent modelling.

Problem Statement

Existing rummy applications are built for monetised competition rather than learning. They never explain why a move is good, so beginners learn only by losing, and their computer opponents are black boxes. The central computation of the game, finding the best arrangement of a hand and measuring how far it is from a valid declaration, is combinatorially large: a 13-card hand contains thousands of candidate melds, and naive search grows exponentially.

Objectives

1. To model the game as an AI environment and each computer player as a rational agent, described in the PEAS framework.
2. To implement a rules engine that validates melds and declarations, using backtracking search for the declaration problem.
3. To design an efficient state-evaluation function, using a memoised bitmask dynamic program for the minimum-deadwood arrangement.
4. To build decision procedures for the draw and discard actions using heuristic simulation, lookahead and exact probability.
5. To create agents at three difficulty levels and a coach that explains each recommendation to the player.
6. To evaluate correctness and performance through automated tests and simulated games.

Scope

The project covers single-human Points Rummy against one to three computer agents, with cumulative scoring over rounds, drop and wrong-show penalties, and a responsive browser interface for desktop and mobile. Online multiplayer, accounts and real-money play are outside the scope.

CHAPTER 2

LITERATURE REVIEW

AI research on card games has concentrated on poker and bridge, but rummy-type games have a growing literature that ranges from learning agents to exact combinatorial solvers. A standard reference for the agent concepts used in this project, including rational agents, the PEAS description, heuristic search and decision-making under uncertainty, is Russell and Norvig (2021). Six works that shaped the design of Smart Rummy are reviewed below, each with its methodology and its main drawback.

1. Kotnik and Kalita (2003): “The Significance of Temporal-Difference Learning in Self-Play Training: TD-Rummy versus EVO-Rummy”

Methodology: Two Gin Rummy agents learn a neural-network hand evaluator purely through self-play, one by temporal-difference reinforcement learning and one by an evolutionary hill-climbing method, and are then compared against each other.

Drawback: The learned evaluator is opaque and costly to train, so the agent cannot explain its moves. It addresses Gin Rummy, which has no jokers and no pure-sequence rule.

2. Cowling, Powley and Whitehouse (2012): “Information Set Monte Carlo Tree Search”

Methodology: Extends Monte Carlo Tree Search to imperfect-information games by searching over information sets. Hidden cards are sampled on each iteration, and statistics are shared across the samples.

Drawback: Thousands of simulations are needed per move, which is expensive on a web server, and its statistical decisions are hard to explain to a learner.

3. Den Hertog and Hulshof (2006): “Solving Rummikub Problems by Integer Linear Programming”

Methodology: Formulates the placement of the maximum value of tiles into valid runs and groups as an integer linear program, solved with a standard solver.

Drawback: Depends on an external solver and models Rummikub tiles, not playing cards with a changing wild joker or a mandatory pure sequence.

4. Van Rijn, Takes and Vis (2015): “The Complexity of Rummikub Problems”

Methodology: Analyses the complexity of meld optimisation and gives a dynamic-programming algorithm that runs in polynomial time when the number of suits and copies is fixed.

Drawback: A theoretical study with no playing agent, opponent model or explanation, and without Indian Rummy’s joker constraints.

5. Goldman et al. (2021): “Evaluating Gin Rummy Hands Using Opponent Modeling and Myopic Meld Distance”

Methodology: A tournament agent evaluates hands by combining card values with a “meld distance” (cards missing from melds) and models the opponent from observed pickups and discards.

Drawback: Built for Gin Rummy competition; it produces moves but no human-readable reasons.

6. Larkin, Collicott and Hiebel (2021): “Modeling Expert Knowledge in a Heuristic-Based Gin Rummy Agent”

Methodology: Encodes expert intuition as parameterised rules for drawing, discarding and knocking, with parameters tuned by grid search through self-play.

Drawback: Evaluated only against a simple baseline; its parameters are tuned rather than derived, and it does not teach a human player.

Comparative Summary

Table 2.1 Comparison of reviewed works with the proposed system

Work	Game	Technique	Exact hand evaluation	Explains moves	Interactive speed
Kotnik and Kalita (2003)	Gin	TD learning, evolution	No	No	Yes
Cowling et al. (2012)	General	ISMCTS	No	No	Costly
Den Hertog and Hulshof (2006)	Rummikub	Integer LP	Yes	No	Solver-bound
Van Rijn et al. (2015)	Rummikub	Dynamic programming	Yes	No	N/A
Goldman et al. (2021)	Gin	Heuristics + opponent model	Partial	No	Yes
Larkin et al. (2021)	Gin	Tuned rules	No	No	Yes
Smart Rummy	Indian 13-card	Bitmask DP, simulation, probability	Yes	Yes	Yes

Research Gap

Three gaps emerge from the review. Almost all rummy agents target Gin Rummy, so the Indian variant, with its round-specific wild joker and mandatory pure sequence, is largely unaddressed. The exact optimisation methods from the Rummikub literature have not been combined with a playing agent and an interactive interface. None of the reviewed systems explains its decisions to a human. Smart Rummy addresses all three by applying an exact memoised dynamic program to Indian Rummy hands, building utility-based agents on it, and turning every computation into plain-language advice.

CHAPTER 3

PROPOSED SYSTEM

The proposed system is a web-based Indian Rummy game in which every computer player is an intelligent agent built from classical AI techniques, and in which the same reasoning powers a coach for the human player. The game state lives on a Python server, which exposes it to a React client through a REST API. The algorithms are pure functions over card lists, so the agents, the hints and the automated tests all share exactly the same logic.

Task Environment (PEAS)

Following the agent framework of Russell and Norvig (2021), the task environment of a computer player is specified in Table 3.1.

Table 3.1 PEAS description of the rummy agent

Component	Description
Performance measure	Declare a valid hand first; otherwise keep deadwood points as low as possible; minimise cumulative score over rounds
Environment	Two-deck 106-card shoe, stock and discard pile, wild-joker rank, 2–4 players taking turns
Actuators	Draw from stock, take the open card, discard a card, declare with a finish card
Sensors	Own hand, top of the discard pile, full discard history, wild-joker rank, opponents' pickups from the discard pile

Properties of the Environment

Table 3.2 Classification of the task environment

Property	Value	Reason
Observability	Partially observable	Stock order and opponents' hands are hidden
Agents	Multi-agent, competitive	Up to three opponents pursue conflicting goals
Determinism	Stochastic	The next stock card is random to the agent
Episodes	Sequential	Each discard changes future options
Dynamics	Static	Nothing changes while an agent deliberates
State space	Discrete	Finite cards, actions and turns

Agent Design

Each bot is a **utility-based agent** (Fig 3.1). Its utility is the negative of a *progress cost*, which measures how far a hand is from a valid declaration. On its turn the agent senses the public state, evaluates the hand with the dynamic-programming partitioner, decides whether to take the open card by one-step lookahead, declares if 13 cards can be grouped, and otherwise discards the card whose removal leaves the best expected hand. The three difficulty levels correspond to three agent designs: Easy behaves like a simple reflex agent, Medium is a utility-based agent, and Hard is a model-based utility agent that also remembers which cards are dead.

BOT AS A UTILITY-BASED INTELLIGENT AGENT

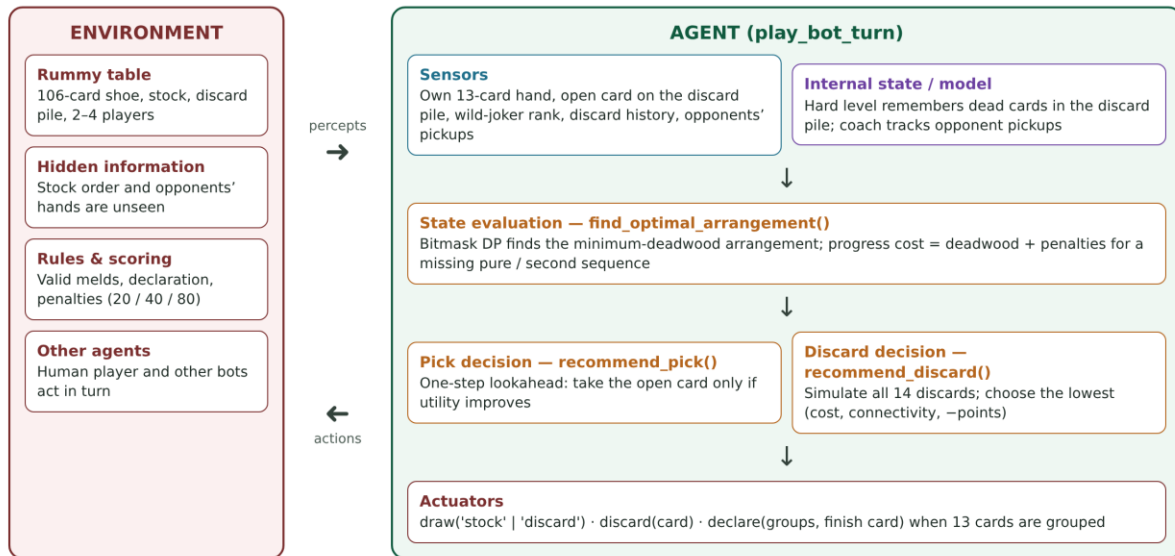


Fig 3.1 Bot as a utility-based intelligent agent

3.1 ARCHITECTURE DIAGRAM

The system follows a layered client–server architecture (Fig 3.2). Each layer depends only on the layer below it, which keeps the AI core independent of the web framework.

- **Presentation layer.** A React and TypeScript single-page application. `App.tsx` runs the human turn, paces bot turns with a 950 ms pause so each move is visible, and grades the player’s moves against the coach. Components render the table, the hand with live meld labels, the coach panel and the dialogs.
- **API layer.** FastAPI exposes thirteen REST endpoints with Pydantic validation and keeps active games in memory, keyed by game id.
- **Game engine.** `RummyGameEngine` holds the players, the deck, the turn state machine, scoring, the event log and the agent policies, and never reveals bot hands to the client.
- **Algorithm (AI) layer.** The rules engine, hand partitioner, assistant and coach: search, dynamic programming, heuristics and probability.
- **Data layer.** The `Card` model and the `Deck`, which builds, shuffles, cuts and deals the 106-card shoe.

Fig 3.3 shows the life cycle of a game from dealing to the results screen, and Fig 3.4 traces one human turn through the layers.

SMART RUMMY — SYSTEM ARCHITECTURE

Three-tier web architecture: React single-page client · FastAPI REST service · deterministic game & algorithm core (no ML)

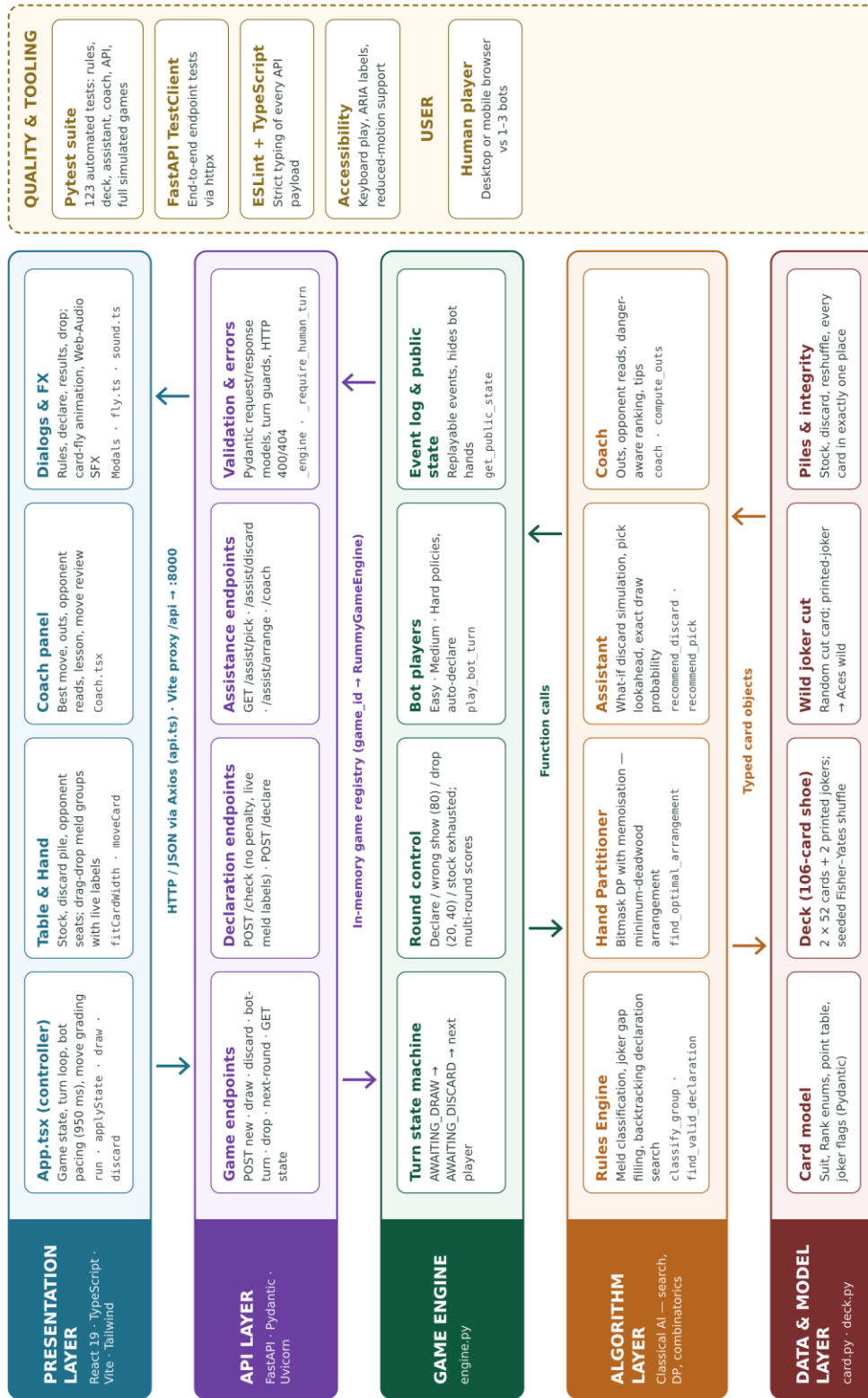


Fig 3.2 Architecture diagram of Smart Rummy

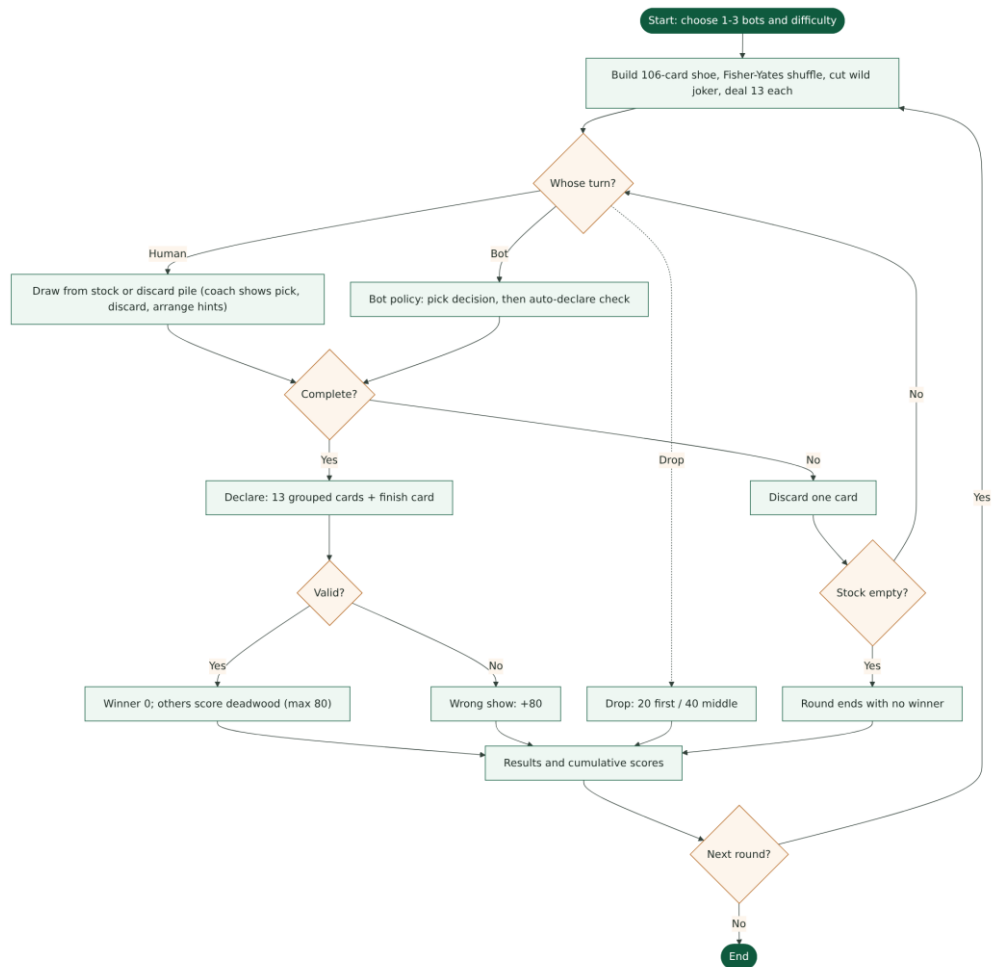


Fig 3.3 Game flow chart

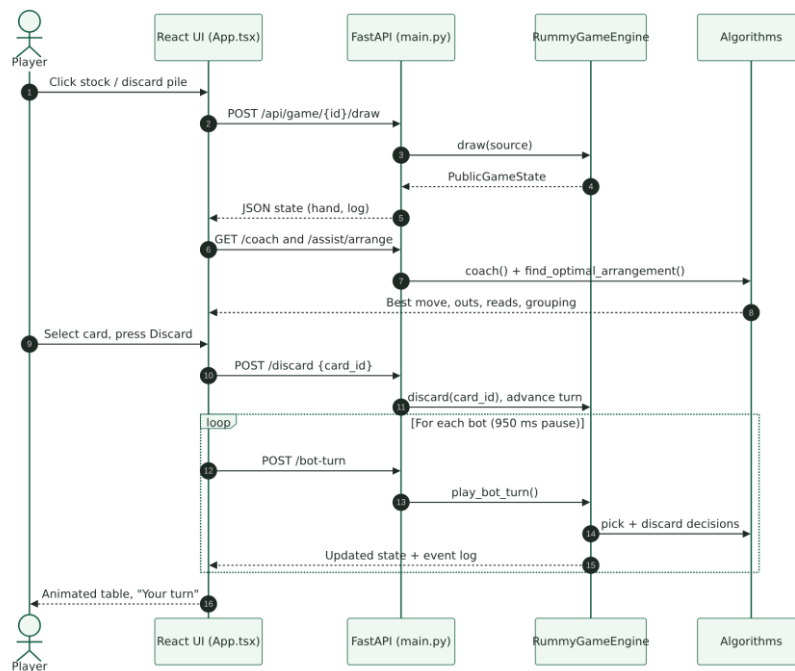


Fig 3.4 Sequence diagram of one turn

CHAPTER 4

MODULES DESCRIPTION

The system is divided into two modules. Module 1 provides the environment: the cards, the deck, the rules and the turn engine. Module 2 provides the intelligence: hand evaluation, action selection, the three agent levels and the coach.

4.1 MODULE 1: GAME ENVIRONMENT AND RULES ENGINE

This module (`card.py`, `deck.py`, `rules.py` and `engine.py`) implements the environment in which the agents act. Its main functions are listed in Table 4.1.

Table 4.1 Main functions of Module 1

Function	Description
<code>Deck.shuffle()</code>	Seeded Fisher–Yates shuffle of the stock, so games can be reproduced
<code>Deck.cut_joker()</code>	Cuts a random card; all cards of that rank become wild (Aces if a printed joker is cut)
<code>Deck.deal()</code> , <code>draw()</code> , <code>discard()</code>	Round-robin dealing; drawing from either pile; reshuffles the discard pile when the stock is empty
<code>classify_group()</code>	Classifies a group as pure sequence, impure sequence, set or invalid
<code>_run_feasible()</code>	Tests whether jokers can fill the gaps of a run, with Ace low or high
<code>validate_arrangement()</code>	Checks a declaration: all groups valid, ≥ 2 sequences, ≥ 1 pure
<code>find_valid_declaration()</code>	Backtracking search for any valid declaration of a hand
<code>RummyGameEngine</code>	Turn state machine, drops (20 / 40), wrong show (80), scoring and event log

Seeded Fisher–Yates Shuffle and Wild Joker Cut

The stock is shuffled with the Fisher–Yates algorithm, which produces each of the $n!$ orderings with equal probability in $O(n)$ time. Each deck owns a seedable generator, so any game can be replayed exactly for testing. A uniformly

random cut then chooses the wild rank; every card of that rank, in both decks, is flagged as a joker worth zero points.

Meld Classification as Constraint Checking

A group is tested from the strictest interpretation to the loosest. A **pure sequence** needs three or more cards of one suit whose ranks are consecutive with the Ace either low (A-2-3) or high (Q-K-A); a wrap-around such as K-A-2 is rejected. For an **impure sequence**, the natural cards are treated as anchors, and `_run_feasible` checks whether a window of the required length exists within legal bounds whose missing positions can be filled by the available jokers. A **set** needs three or four cards of one rank in distinct suits, with jokers filling missing suits.

Declaration as Backtracking Search

Deciding whether a hand can be declared is a constraint-satisfaction problem: partition the cards into valid groups subject to the global constraint of at least two sequences, one of them pure. It is solved by depth-first backtracking with failure memoisation:

```
search(remaining, groups):
  if remaining is empty:
    return groups if (#seq >= 2 and #pure >= 1) else FAIL
  if remaining in unsolvable: return FAIL
  f <- first card of remaining
  for each valid group g of 3..5 cards containing f:
    r <- search(remaining - g, groups + [g])
    if r != FAIL: return r
  unsolvable.add(remaining)           // memoise dead end
  return FAIL
```

Fixing the first remaining card prevents the same partition from being generated in different orders, and memoising unsolvable remainders prunes repeated dead ends.

4.2 MODULE 2: INTELLIGENT AGENT AND COACH

Module 2 (`hand_partitioner.py`, `assistant.py`, `coach.py` and the agent policies in `engine.py`) contains the intelligence of the system. Its main functions are listed in Table 4.2.

Table 4.2 Main functions of Module 2

Function	Description
<code>find_optimal_arrangement()</code>	State evaluation: minimum-deadwood arrangement by memoised bitmask DP
<code>_progress_cost()</code>	Utility: deadwood plus penalties for a missing pure or second sequence
<code>_connectivity_potential()</code>	Heuristic usefulness of a card, reduced by dead neighbours
<code>recommend_discard()</code>	Simulates all 14 discards and ranks them
<code>recommend_pick()</code>	One-step lookahead on the open card
<code>calculate_draw_probability()</code>	Exact probability of drawing a useful card
<code>compute_outs()</code> , <code>opponent_reads()</code>	Coach: helpful unseen cards; opponents' likely plans
<code>play_bot_turn()</code>	Runs one agent turn according to its difficulty level

State Evaluation by Bitmask Dynamic Programming

Arranging a hand is a weighted set-packing problem: choose non-overlapping melds that cover the most points. All valid melds of three to five cards are enumerated first and indexed by their lowest card. The search state is $(mask, pure, seqs)$, where *mask* marks decided cards and *pure* and *seqs* count the pure and total sequences chosen so far. At each state the lowest undecided card is either left as deadwood or placed into a meld that contains it and does not overlap used cards. Each card is decided exactly once and every state is memoised, so the search stays small (Fig 4.1). A constrained pass enforces the declaration rule; if it fails, an unconstrained pass finds the most useful grouping and the hand is scored at full count, capped at 80.

```

go(mask, pure, seqs):
  if mask == ALL: return 0 if structure ok else -INF
  if (mask, pure, seqs) in memo: return memo[...]
  i <- lowest undecided card
  best <- go(mask | bit(i), pure, seqs) // i is deadwood
  for meld m containing i, m disjoint from mask:
    best <- max(best, points(m) + go(mask | m, pure', seqs'))
  memo[(mask, pure, seqs)] <- best
  return best

```

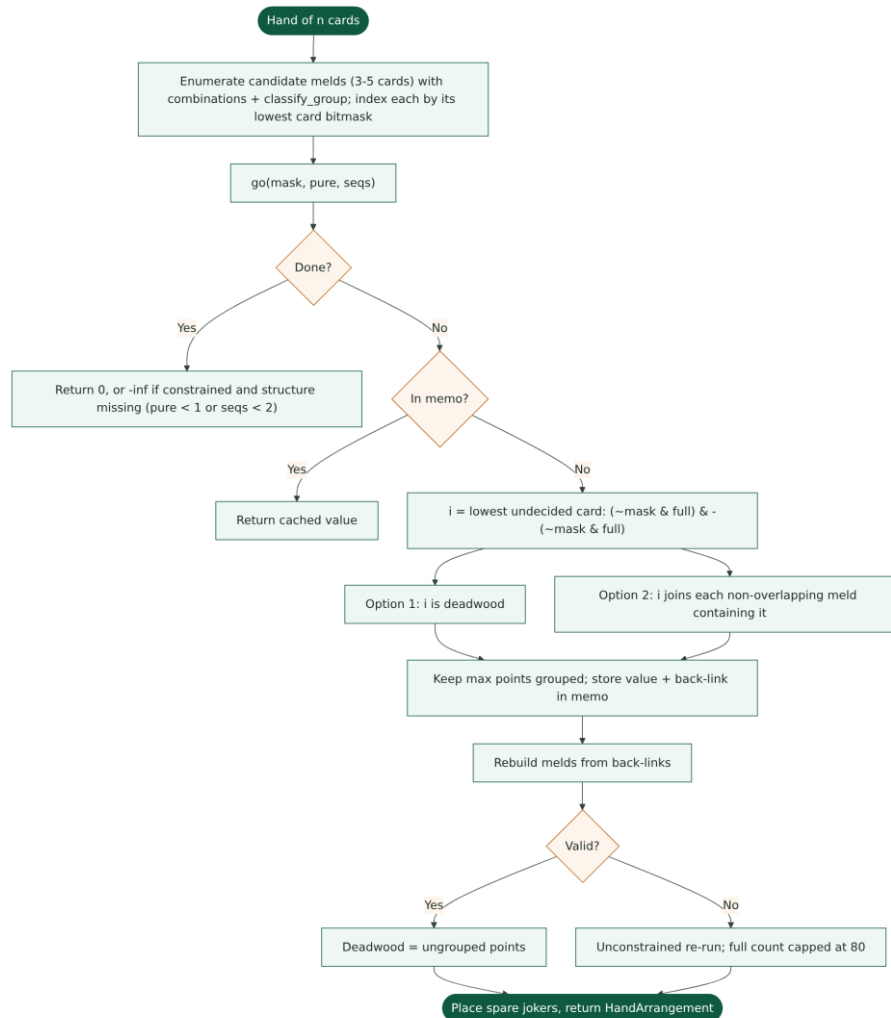


Fig 4.1 Flow of the hand partitioner (bitmask DP)

Action Selection by Discard Simulation

`recommend_discard` treats the discard as a one-ply search over 14 actions. For each card it removes the card, evaluates the remaining 13 with the partitioner, and ranks the outcomes lexicographically by (progress cost, connectivity, –points). Candidate melds are enumerated once and reused in all 14 evaluations. Discarding a joker is heavily penalised. The top three options are returned with a score and a sentence of reasoning (Fig 4.2).

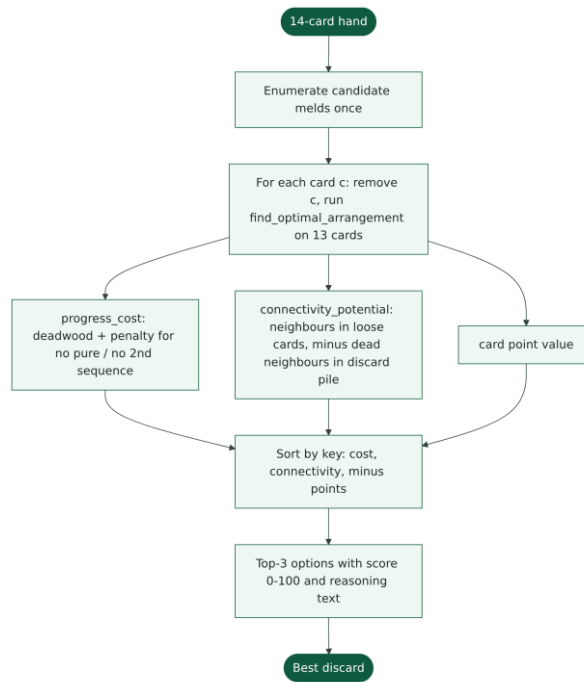


Fig 4.2 Discard recommendation pipeline

Heuristics, Lookahead and Probability

- **Progress cost (utility).** It equals the true deadwood once a pure sequence and a second sequence exist; otherwise fixed penalties are added for each missing requirement, because the whole hand counts in that case.
- **Connectivity heuristic with card counting.** A card scores 25 for each neighbour among the loose cards (same rank in another suit, or same suit within two ranks), capped at 100, minus 10 for every neighbour whose two copies are already in the discard pile.
- **One-step lookahead.** The open card is taken only if it would be kept and the progress cost strictly decreases.
- **Exact probability.** The chance that the next stock card helps is the number of useful unseen copies divided by the number of unseen cards (106 minus the cards seen).

Agent Levels

Table 4.3 Agent designs for the three difficulty levels

Level	Agent type	Pick decision	Discard decision
Easy	Simple reflex	Take the open card only if it is a joker	Highest-value card not in a meld
Medium	Utility-based	recommend_pick (lookahead)	recommend_discard (simulation)
Hard	Model-based utility	recommend_pick	recommend_discard with memory of dead cards

Every agent runs the partitioner after drawing and declares when 13 cards are fully grouped with one card left for the finish slot.

Coach: Outs and Opponent Modelling

The coach turns the agent's reasoning into advice for the human. **Outs** are found by adding each candidate unseen card to the hand as a probe and keeping those that lower the progress cost; they are listed with the number of copies left. **Opponent reads** come from each opponent's pickups from the discard pile: the coach warns against discarding cards near them, and when two discards are equally good it ranks second the card the next player is likely to want. Each human move is then graded as *best*, *close* or *better option*.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The back end was implemented in Python with FastAPI and the front end in React with TypeScript, built with Vite. Table 5.1 lists the environment used for development and experiments.

Table 5.1 Hardware and software environment

Item	Specification
Processor / Memory	Intel Core i5 (or equivalent), 8 GB RAM
Operating system	Windows 11 (development); Linux (experiments)
Back end	Python 3.12+, FastAPI, Pydantic v2, Uvicorn
Front end	React 19, TypeScript, Vite, Tailwind CSS, Axios
Testing	Pytest with FastAPI TestClient; ESLint
Browser	Chromium / Chrome (desktop 1440 × 900 and mobile 390 × 844)

Three kinds of experiment were run:

1. **Correctness.** 123 automated Pytest cases covering the deck, the rules, every algorithm, the REST API and complete simulated games.
2. **Performance.** The partitioner, the discard simulation and the declaration search were timed on 200 randomly dealt hands (seeds 0–199), and every agent turn was timed during simulated games.
3. **Agent strength.** For each difficulty, 15 seeded rounds were played with two agents and a scripted player, recording the number of turns until an agent declared.

Screenshots were captured automatically with Playwright while playing real games through the user interface.

5.2 RESULTS

Correctness

All 123 tests pass. Table 5.2 summarises the suite.

Table 5.2 Automated test results

Test file	Area covered	Tests	Result
test_rules.py	Sequences, sets, Ace handling, declarations, backtracking	40	Pass
test_deck.py	Shoe composition, points, shuffle, joker cut, dealing, reshuffle	47	Pass
test_assistant.py	Discard and pick advice, reasoning, probability	17	Pass
test_api.py	Turn flow over HTTP, wrong show, drop penalty	4	Pass
test_hints.py	Hints never break a meld or throw a joker	4	Pass
test_algorithms.py	Winning hands and deadwood scoring	3	Pass
test_engine.py	Initialisation, turns, agent automation	3	Pass
test_full_games.py	Full games to completion; declaration scoring	3	Pass
test_coach.py	Coach flow and “declare” advice	2	Pass
Total		123	All pass

Performance and Agent Strength

Table 5.3 and Fig 5.1 report the measurements. Every operation finishes well below the 950 ms pause between bot moves, so the game feels instant. The utility-based agents (Medium and Hard) needed on average 26 turns to declare, against 38.5 for the reflex agent (Easy). This confirms that the heuristic evaluation produces markedly stronger play.

Table 5.3 Measured performance

Operation	Mean	95th percentile
Hand partitioner (13 cards)	22.6 ms	29.8 ms
Discard simulation (14 cards)	39.7 ms	61.0 ms
Declaration search (backtracking)	63.0 ms	157.2 ms
Easy agent turn	37.5 ms	73.4 ms
Medium agent turn	151.2 ms	236.9 ms
Hard agent turn	146.9 ms	258.6 ms

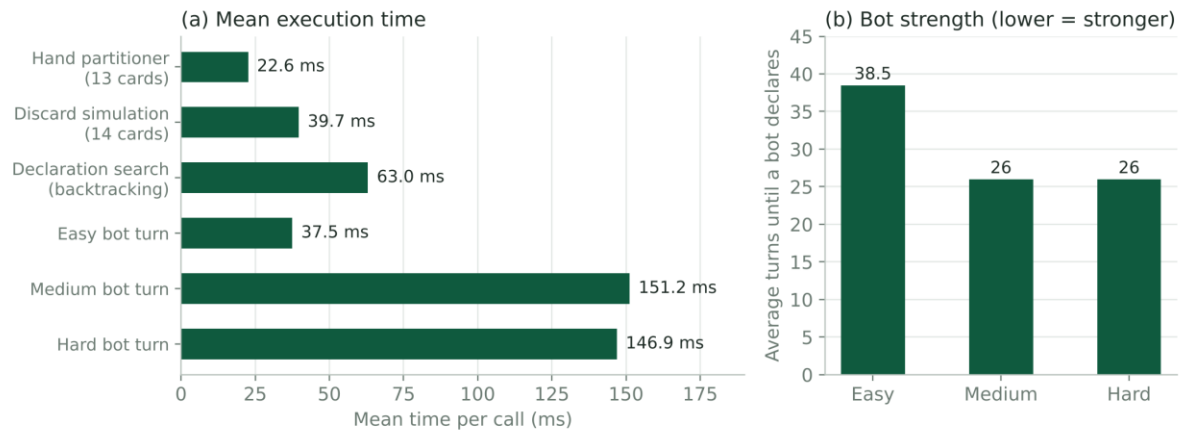


Fig 5.1 Execution time and agent strength

Output Screenshots

Figs 5.2 to 5.10 show the application during real games.

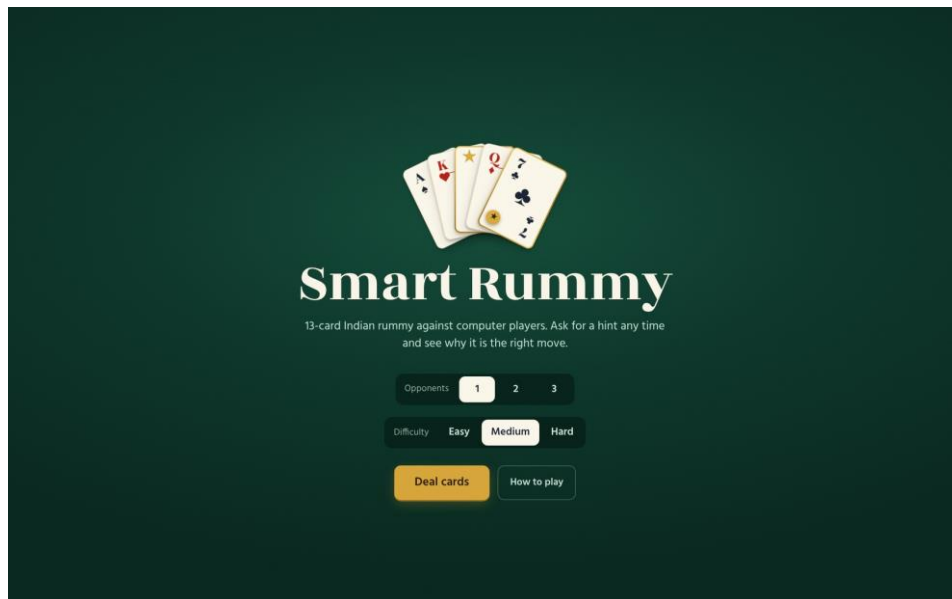


Fig 5.2 Start screen with opponent and difficulty selection



Fig 5.3 Game table with the coach panel and helpful cards (outs)



Fig 5.4 Auto-arranged hand with live meld labels



Fig 5.5 Discard recommendation with reasoning and move review



Fig 5.6 Mid-game view with opponent reads and agent moves

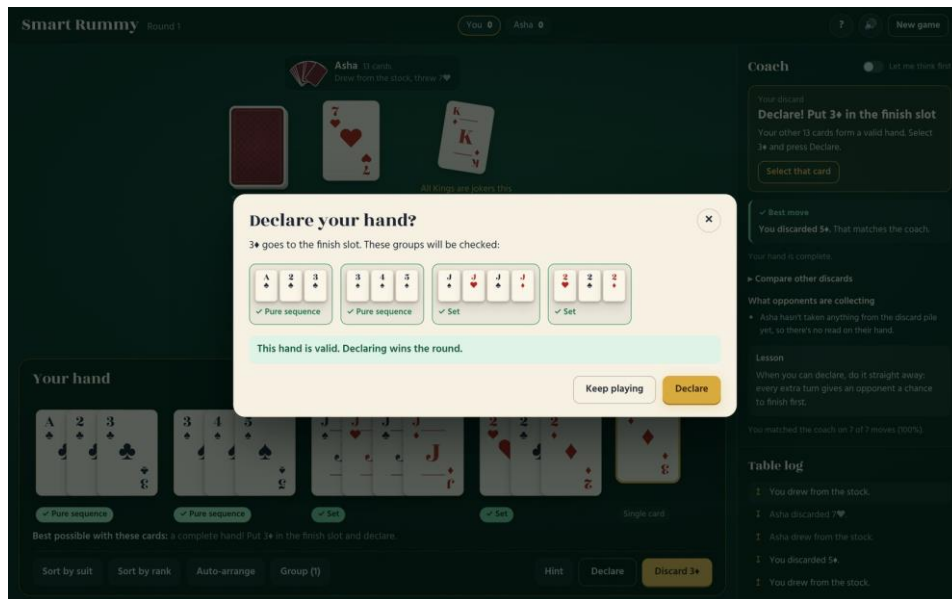


Fig 5.7 Declare dialog validating every group

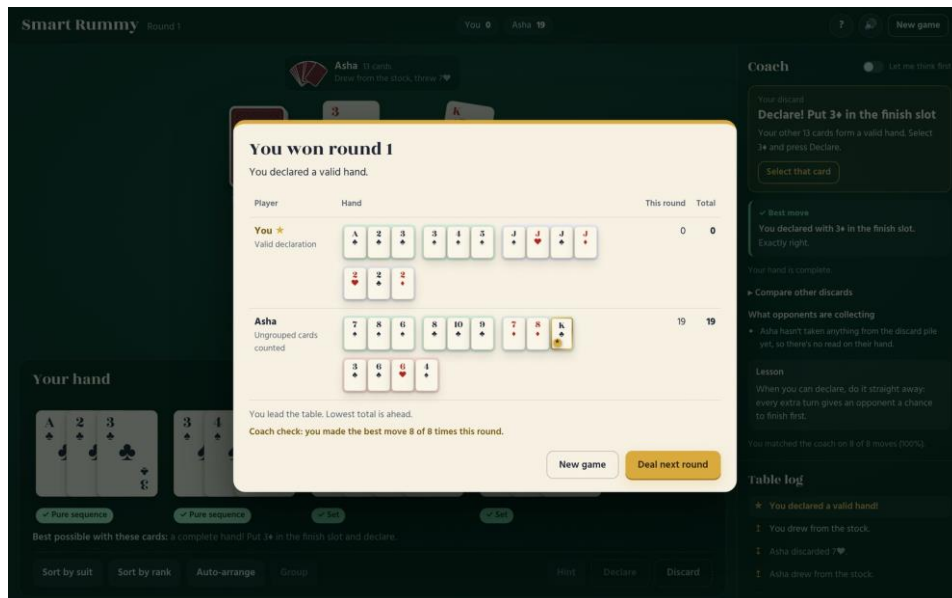


Fig 5.8 Round result after a valid declaration

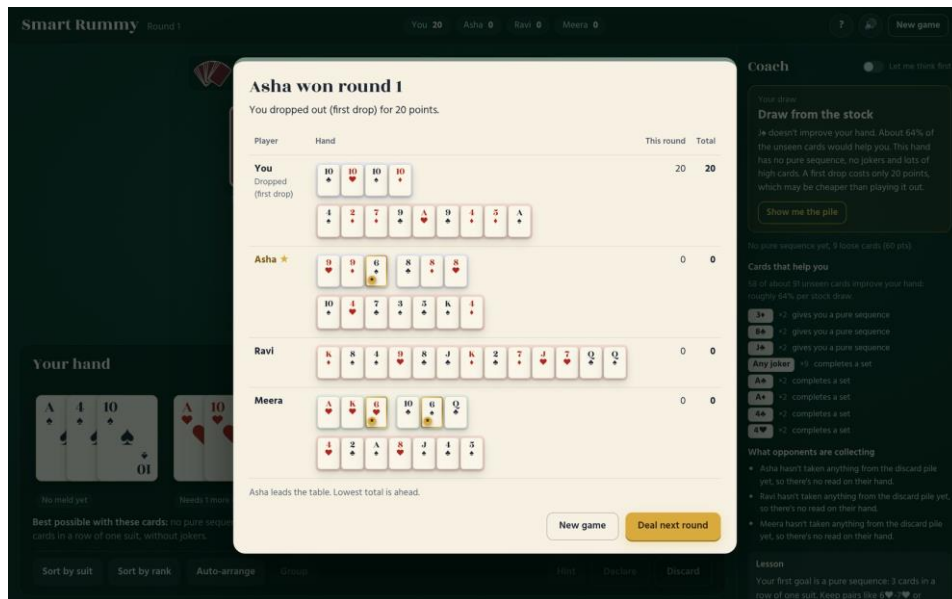


Fig 5.9 Round result after a first drop (20 points)

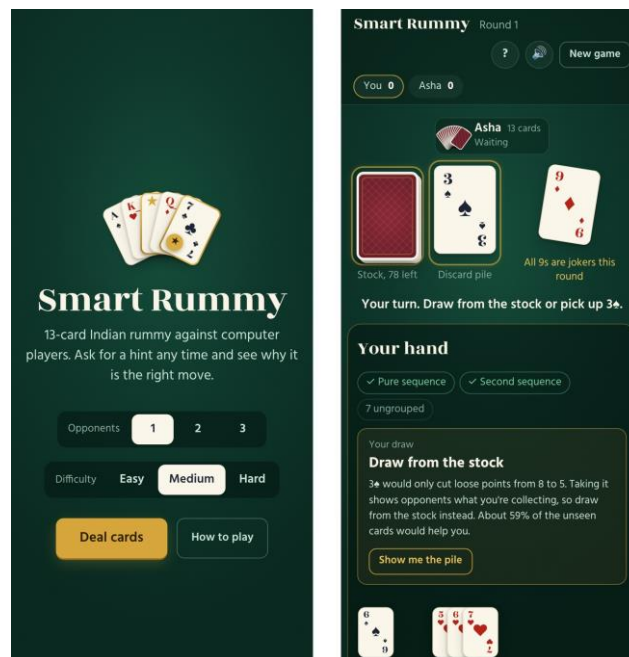


Fig 5.10 Responsive layout on a mobile phone

CHAPTER 6

CONCLUSION AND FUTURE WORK

Conclusion

Smart Rummy shows that classical AI techniques are enough to build capable, fast and explainable agents for a partially observable, multi-agent card game. Backtracking search verifies declarations, and a memoised bitmask dynamic program evaluates any hand in about 23 ms. Heuristic discard simulation, one-step lookahead and exact probability let utility-based agents play at three distinct levels of strength. Because every decision comes from a traceable computation, the same reasoning drives a coach that tells the player which move is best and why, which cards would help, and what the opponents seem to be collecting. The system passes 123 automated tests and meets every objective set out in Chapter 1.

Future Work

- An “Expert” agent based on Information Set Monte Carlo Tree Search, using the dynamic program as its evaluation function.
- A learned opponent model that estimates hidden hands from the full history of picks and discards.
- Online multiplayer over WebSockets, with agents filling empty seats.
- Pool Rummy (101 / 201) and Deals Rummy variants.
- User accounts with long-term progress tracking, and Tamil and Hindi versions of the coach.

CHAPTER 7

REFERENCES

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*, 4th edn, MIT Press, Cambridge, MA.

Cowling, P.I., Powley, E.J. and Whitehouse, D. (2012) ‘Information Set Monte Carlo Tree Search’, *IEEE Transactions on Computational Intelligence and AI in Games*, Vol.4, No.2, pp.120-143.

Den Hertog, D. and Hulshof, P.B. (2006) ‘Solving Rummikub Problems by Integer Linear Programming’, *The Computer Journal*, Vol.49, No.6, pp.665-669.

FastAPI (2026) *FastAPI Documentation*, available at: <https://fastapi.tiangolo.com/>

Goldman, P., Knutson, C.R., Mahtab, R., Maloney, J., Mueller, J.B. and Freedman, R.G. (2021) ‘Evaluating Gin Rummy Hands Using Opponent Modeling and Myopic Meld Distance’, *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol.35, No.17, pp.15510-15517.

Knuth, D.E. (1997) *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd edn, Addison-Wesley, Reading, MA.

Kotnik, C. and Kalita, J. (2003) ‘The Significance of Temporal-Difference Learning in Self-Play Training: TD-Rummy versus EVO-Rummy’, *Proceedings of the 20th International Conference on Machine Learning (ICML)*, Washington, DC.

Larkin, S., Collicott, W. and Hiebel, J. (2021) ‘Modeling Expert Knowledge in a Heuristic-Based Gin Rummy Agent’, *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol.35, No.17, pp.15577-15582.

React (2026) *React Documentation*, available at: <https://react.dev/>

Russell, S. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*, 4th edn, Pearson, Hoboken, NJ.

Van Rijn, J.N., Takes, F.W. and Vis, J.K. (2015) ‘The Complexity of Rummikub Problems’, *Proceedings of the 27th Benelux Conference on Artificial Intelligence (BNAIC)*, Hasselt, Belgium.